

Case 2

02582 Computational Data Analysis

Group 14 - s224766, s224381, s224360 & s215141

May 3, 2026

Introduction and Data description

This task consisted of an exploratory and unsupervised analysis of the dataset EmoPairCompete **datasource**. EmoPairCompete is based on biosensory data from participants of an emotional study, measuring the stress levels of different participants during a puzzle exercise. Additionally to the biosignal data (including heart rate, blood volume pulse (BVP), electrodermal activity (EDA) and temperature) the participants also answered a range of questions after each phase of the experiment, to measure their levels of stress. Exactly these responses and biosensor measures are what we aim to gain an understanding of, how does the body and mind react in situations of high pressure and competition? This concludes in the question, determine if a person is in a stressful situation through quantification of their mind and body?

We need to update the aim when frederiks stuff is done :DD

I think we in the introduction need to answer the following questions: Formulate a clear purpose for your project, so the aim of your analysis is clear. What are your research questions? What kind of representation would you like to derive from the features of the biosignals? What could it be used for? We expect you to use methods from the course curriculum.

Data overview

The provided dataset includes the raw data as well as two preprocessed dataframes, one `HR_data.csv` and the updated `HR_data2.csv`. The raw data consists of data from 6 different days and 26 different people, where each person went through 4 rounds of experiments, where each round is split in three phases. The three phases were a pre-puzzle phase, a during puzzle phase and a post puzzle phase, where during each the biosensor measured heart rate, BVP, EDA and temperature. After each phase a questionnaire was answered, reporting the feelings of each person, including frustrated, upset, hostile, alert, ashamed, inspired, nervous, determined, attentive, afraid, active and lastly how difficult they found

the task.

Each day has a `team_info.csv` file, indicating the teams of 2, and who is the puzzler in each team.

To maintain full control over the data pipeline, we opted to work directly with the raw data rather than the preprocessed versions provided through the course material.

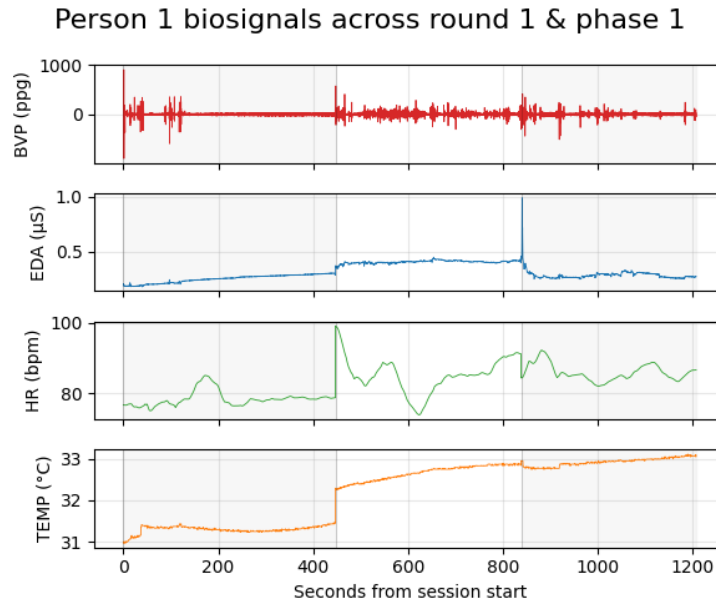


Figure 1: On the plot is shown the raw data of the four different parameters measured by the biosensor. In this case it is the first person (day 1, person 1) through the first round and all three phases.

Preprocessing

Collecting personal data

To utilize the raw time series data we have collected all the biosensor data in one dataframe per person labelled `Person{i}_{DayID}_{PersonID}.csv` where $i \in \{1, \dots, 26\}$ is the number of participants with usable data. This means that all rounds and phases of each person were combined, with the column `time` being the primary key, as one person can only be one place at a time.

The questionnaire responses we also collected for each round and phase in dataframes named `Person{i}_{DayID}_{PersonID}_responses.csv`. Lastly, to ensure systematic oversight of the data, a dictionary of information of each person was created. This dictionary included start and end timestamps of all rounds and phases, as well as group ID and if they were a puzzler or not. WE DONT END UP USING THIS, REMOVE?

Normalization and extraction of features

After preprocessing the raw biosignal data into individual frameworks, we normalize the data for each person using exclusively their 'Phase1' baseline period. Then, using a sliding window each persons data is aggregated into 30 second intervals, in a manner that prevents a given window to overlap from one phase or round into the next (in case of any remaining <30 seconds in the phase after the aggregation we would consider this as one smaller window). For each 30 second segment we extract the mean, standard deviation, min and max values for the four biosignals, and additionally the peak counts for EDA and slope for TEMP. By discarding the "HR_data" files, working directly with the raw temporal domain data, applying the 30 seconds rolling windows and extracting our own features we are satisfying the advanced challenge: Temporal Dynamics & Feature Engineering. Normalizing by the baseline for each person before extracting features rather than the other way around means we also account for the per-participant differences such as resting body temperature or resting heart rate.

Model and method

One-class support vector machine

We choose to analysis the bio signal and response data from a **Zero-Shot Stress Detection** point of view, where we employ an unsupervised One-Class Support Vector Machine (OC-SVM) Model. We train the model solely on the 'Resting' phase with the objective of being able to label any other phase ('puzzling' and 'post-puzzling ') as anomalies.

We pick a SVM model as we expect to see a great deal of overlap in the bio-signals across phases. We therefore opt for a model that is adaptable to this challenge, i.e. SVMs. Secondly the use of SVM allows us to use the Kernel trick. The Kernel Trick allows a model to operate in a high-dimensional feature space without ever explicitly calculating the coordinates of the data in that space. Instead of performing a basis expansion $h(X)$, we define a Kernel function $\mathcal{K}(x_i, x_j)$ that represents the inner product of those expanded features: $\mathcal{K}(x_i, x_j) = \langle h(x_i), h(x_j) \rangle$. In other worrds the kernel trick allows us to calculate geometric relationships in infinite dimensions without actually mapping the data there.

[source here?](#)

We make use of the RBF kernel given by: $\mathcal{K}(x, x') = e^{-\gamma \|x - x'\|^2}$ which mathematically represents a dot product in an infinite-dimensional space. We choose this kernel as it allows us "form a none linear relationships" and the biosignals are none linear ([we think??](#)) [source](#)

One-Class SVM deviates from standard SVP by instead of separating N classes from each other, One-Class SVM defines a single 'normal class' from data and is then able to single out anomaly inputs that does not fit within this class. This makes it an ideal model for stress detection.

There are some downsides related to this model choice: **what is the purpose of the points below? should they stay?**

- Kernel methods do not scale well. Limited to around 10,000-20,000 observations
- Kernel methods generally do not do variable selection in any reasonable or automatic way
- With more features than observations there is always a separating hyper-plane
- Actually infinitely many which we have to choose between
- Potential problem with large number of features if many of them are garbage
- SVM does not generalize gracefully when the number of classes are more than two
- Frequently used for multiclass classification anyway

Gaussian Mixture Model (GMM)

The second model we consider is a Gaussian Mixture Model (GMM). The basic idea is to assume that our n -dimensional feature vectors (where n is the number of features extracted per window) are drawn from a weighted sum of K multivariate Gaussian distributions. So instead of assuming the resting data follows a single bell curve in n -D space, we assume it can be described by K such bell curves, each with their own mean μ_k and covariance Σ_k . Formally the density is

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k), \quad \sum_{k=1}^K \pi_k = 1, \quad (1)$$

where π_k is the mixing weight (the prior probability of component k). For our anomaly detection use-case the idea is that resting windows fall in regions where $p(x)$ is high, while puzzle windows ideally fall in low-density regions, so we use $-\log p(x)$ directly as the anomaly score.

The parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ are fit by maximum likelihood, but the likelihood does not have a closed form solution because we do not know which Gaussian generated each point. The standard trick is the "Expectation-Maximization" (EM) algorithm, which sets up the problem as an iterative optimization. We introduce a variable γ_{ik} , which is the posterior probability that data point x_i was generated by component k . EM then alternates between two steps:

- **E-step:** given the current parameters, update γ_{ik} by

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}. \quad (2)$$

- **M-step:** with the updated γ_{ik} , update μ_k , Σ_k and π_k as their weighted averages over the data. Concretely, letting $N_k = \sum_{i=1}^N \gamma_{ik}$, we update by

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} x_i, \quad (3)$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top, \quad (4)$$

$$\pi_k = \frac{N_k}{N}. \quad (5)$$

The two steps each increase the likelihood, so the algorithm is guaranteed to converge to a local maximum, but not a global one. The final solution therefore depends on the initialization, and different starts can land in different local optima. The scikit-learn implementation combats this by using k-means clustering on the training data first and then seeding EM with the resulting cluster centers [1].

The remaining choice is the number of components K . We pick K by minimizing the Bayesian Information Criterion (BIC) on the inner LOSO loop, which trades off log-likelihood against model complexity and avoids overfitting on the training subjects, as we saw in week 2 of the course.

Model selection

To perform strictly the "zero-shot" anomaly detection we employ a Leave-One-Subject-Out (LOSO) Cross-Validation scheme. This is where we iterate through each of the 26 subjects, isolating them one-by-one, training the GMM and SVM model only on the Resting, "Phase1" of the remaining 25 subjects and then use this trained model to detect how anomalous each phase of the isolated subject is. Crucially, the models are only trained and tuned on Phase 1 data. Prior to the modeling part we use 'StandardScaler' to scale all the extracted features to zero their means and make their standard deviations one. We then perform dimensionality reduction using Principal Component Analysis retaining components that explain 95% of variance. Both the scaling and dimensionality reduction are performed within the cross-validation loop preventing data leakage. For the OC-SVM model, ν and γ parameters are tuned on the inner loop subjects, the parameters explored were $\nu \in [0.05, 0.10, 0.15, 0.20, 0.25]$ and $\gamma \in [\text{scale}, \text{auto}, 0.1, 0.01]$. Where the optimal combination is chosen by finding the lowest Calibration Error on the inner validation subjects (i.e. if it is $\nu = 0.10$, then the best model would classify exactly 10% of data as anomalous). The GMM model was also tuned on inner loop subjects, here minimizing the Bayesian Information Criterion (BIC) for different numbers of distributions. To ensure a fair comparison, we applied the empirically tuned baseline allowed outliers ν from the SVM model to also be the percentile threshold for the GMM model.

THINGS TO THINK ABOUT:

- is the ν and γ the same for both models? look into course material
The kernel used, is it okay to use in both of them

Missing values

Biosensor data

Working with the raw datafiles meant dealing with missing values. We checked for missing values in all the biosensor data, that is the `BVP.csv`, `TEMP.csv`, `HR.csv` and `EDA.csv` files, but did not find any missing entries. Our assumption is that this is a product of the way the E4 biosensor collects and saves data. That is that if there is a 'missing value' in the recording, then the nature of for example how HR is measured, the recording will smooth over that. Other than the absence of missing values in the dataframes themselves, the time-intervals between each timestamp was also identical within all the datasets.

Response data

The response files had unlike the biosensor data instances of missing values. That is missing responses within some of the columns of the questionnaire that participants had to answer. There are missing values in 25 of the 312 files, one file with 4 missing entries, and 24 with only one missing entries. Most missing values (75%) were in the category *difficulty*, and was only missing in the pre and post puzzle ling phase except for once in the puzzling phase. This however is expected as the participants were only supposed to assess *difficulty* in phase 2, and since we aim to separate the phase 1 from 2 and 3, this column becomes negligible.

Since we have opted to only use the biosensor data in as a training basis, the missing values in the response sheets do not create any issues for the model. We do however compare the responses with the corresponding average anomaly rating for each phase of each person. This is done as a way to evaluate and perspective our model to the personal responses of each person, and if missing values occur, then we simply do not compare those observations. We have chosen not to use imputation to fill in the gaps as the data is categorical, and though we could add an extra category of 'missing' it was decided that it would not add any value in our user case.

Factor handling

As mentioned in the Missing values section, we are working with categorical response data, namely discrete values from either 0-5 or 0-10. All categorical classes had 'bad' ratings at 0, and 'good' at 5 or 10 according to the range. Though the information is categorical, it is clearly ordinal, and therefore we decided to normalize them using means and standard deviations (std) from

each response class and person. We subtract mean to combat the bias of the general tendency of rating, that is if a person tends to rate generally high or low. A bias in the personal range of ratings can also skew the data, which is combated by the division with std.

- Med kategoriske data normaliseret som vi gør er vi mere prone til at få værdier på 0, hvis participans ikke ændrer deres var mellem phases, men i den phase skulle vi også gerne ikke se nogen høj anomaly

The puzzler id (we dont use before the review with responses), responses are categorical but we currently normalize them by subtracting the mean

Model validation

This is how we performed our model validation and estimated the EPE

Results

To evaluate how well the models separate rest from puzzle we use the AUROC (Area Under the Receiver Operating Characteristic curve) as our main metric. Each model gives every 30 second window an anomaly score, and the ROC-curve is obtained by sweeping a threshold over those scores and plotting the true positive rate against the false positive rate. The AUROC is then the area under that curve. A nice property is that it has a direct probabilistic interpretation: it is the probability that a randomly drawn puzzle window gets a higher anomaly score than a randomly drawn resting window. With $\mathcal{P}$ denoting the set of puzzle windows and $\mathcal{R}$ the resting windows, this can be written as

$$\text{AUROC} = \frac{1}{|\mathcal{P}||\mathcal{R}|} \sum_{i \in \mathcal{P}} \sum_{j \in \mathcal{R}} (\mathbb{1}_{\{s_i > s_j\}} + \frac{1}{2} \mathbb{1}_{\{s_i = s_j\}}), \quad (6)$$

where s_i is the anomaly score for window i and $\mathbb{1}_{\{.\}}$ is the indicator function (1 if the condition holds, 0 otherwise). The reason it takes exactly this form is that it is the empirical version of the probability $\mathbb{P}(s_i > s_j)$ for a randomly drawn puzzle window i and resting window j : we are simply looping over all possible (i, j) pairs, counting how often the puzzle window scores higher, and dividing by the total number of pairs $|\mathcal{P}||\mathcal{R}|$. Ties are split in half so a model that gives every window the same score gets exactly 0.5, matching the chance baseline. So a value of 1.0 means perfect ranking, 0.5 is random guessing, and below 0.5 means the model ranks the wrong way around.

We ran both models through the LOSO protocol described above, and tried three different feature setups: the raw 30 second feature vector (*none*), a PCA projection keeping 95% of the variance (*var95*), and a 2D PCA projection (*2d*). For each setup we report the mean per-subject AUROC and the pooled AUROC across all 26 held-out subjects in table ???. The mean AUROC is just the average of the per-subject scores, while the pooled AUROC is computed

by stacking all the anomaly scores and labels from every fold and computing one ROC-curve over the lot. The two numbers tell us slightly different things: the mean treats each person equally, while the pooled score is closer to "if we deployed this model on the whole population, how well would it rank rest vs. puzzle on average".

Model	PCA	Mean AUROC	Pooled AUROC
OC-SVM	none	0.470	0.481
OC-SVM	var95	0.481	0.491
OC-SVM	2d	0.593	0.585
GMM	none	0.626	0.622
GMM	var95	0.598	0.595
GMM	2d	0.585	0.582

Table 1: LOSO performance for both models across the three PCA configurations. Higher is better, 0.5 is chance.

The GMM clearly does better than the OC-SVM baseline. On the full feature space the OC-SVM is basically guessing (mean AUROC 0.470), while the GMM gets a mean of 0.626 and a near-identical pooled AUROC of 0.622. Interestingly the two models react to PCA in opposite ways: squashing the features down to 2D actually helps the OC-SVM a lot ($0.470 \rightarrow 0.593$), while the same projection hurts the GMM ($0.626 \rightarrow 0.585$). Our interpretation is that the RBF-kernel SVM is sensitive to noisy or redundant features, so cutting down to 2 components effectively denoises the input. The GMM on the other hand actually uses the extra variance, so dropping components throws away useful structure. Based on this we pick the **GMM with no PCA** as our final model.

Per-subject behaviour

The aggregate AUROC numbers hide a lot of variation between subjects. For our chosen model the per-subject AUROCs range from 0.43 (Person19) to 0.79 (Person9), with 21 out of 26 subjects above chance and 5 below. The model works best on Person9 (0.79), Person10 (0.74), Person17 (0.72), Person25 (0.72) and Person21 (0.71), and worst on Person19 (0.43), Person5 (0.46) and Person2 (0.48). The standard deviation across subjects is around 0.10, which means that even though the average looks decent, the model can still fail on a new person. This spread is really the dominant uncertainty in our EPE estimate, and is something we come back to in the discussion.

Flagging rate per phase

Beyond AUROC (which only cares about ranking) it is useful to look at how often each model actually flags a window as anomalous. Figure ?? shows,

for each experimental phase, the percentage of windows that ended up above the model's anomaly threshold, averaged across all 26 subjects (with error bars showing the spread between subjects). Both models flag around 16-18% of the resting windows in P1, which is roughly what we would expect since the threshold is calibrated on resting data. The flagging rate then jumps to about 30% in P2 (puzzle), confirming that the puzzle phase looks meaningfully different from rest to both models. The interesting bit is P3 (recovery): the SVM actually flags *more* windows here than during the puzzle (around 36%), while the GMM stays at the puzzle level. One explanation is that the body does not fully return to baseline immediately after the puzzle, i.e. heart rate and EDA decay over time, so what we labelled as "recovery" is biosignal-wise still elevated. The error bars are large though, so we should not over-interpret the difference between the two models.

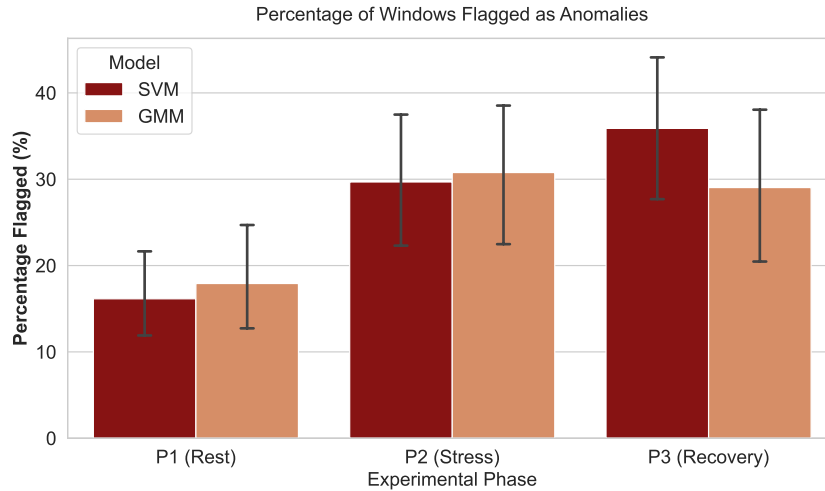


Figure 2: Average percentage of 30 second windows flagged as anomalous, broken down by experimental phase and model. Error bars indicate the standard deviation across the 26 subjects.

Anomaly scores vs. self-reported emotions

A natural sanity check is to ask whether the windows the model flags as anomalous correspond to people actually feeling stressed. We do this by computing, for each subject, the change in mean anomaly score between P1 and P2 ("anomaly jump") and correlating it against the change in their self-reported emotion ratings between the same two phases. Figure ?? shows this for three of the response categories, *Active*, *Nervous* and *Frustrated*, with both models on separate rows.

The strongest correlation is between the GMM anomaly jump and the change in reported *Active* ($r = -0.42$, $R^2 = 0.18$). The negative sign is interest-

ing: subjects whose biosignals look more anomalous during the puzzle actually report feeling *less* active. One way to read this is that the model is picking up on a sympathetic nervous system response (high HR, high EDA) that the subjects themselves perceive as being drained or tense rather than energized. *Nervous* shows a weak positive trend ($r = +0.23$ for GMM), in the direction one would expect, but with a tiny R^2 it is not really statistically meaningful. *Frustrated* is essentially flat for both models, suggesting that frustration is decoupled from the autonomic signals we measure. Across the board the GMM correlations are stronger than the SVM ones, which is consistent with the AUROC results in table ??.

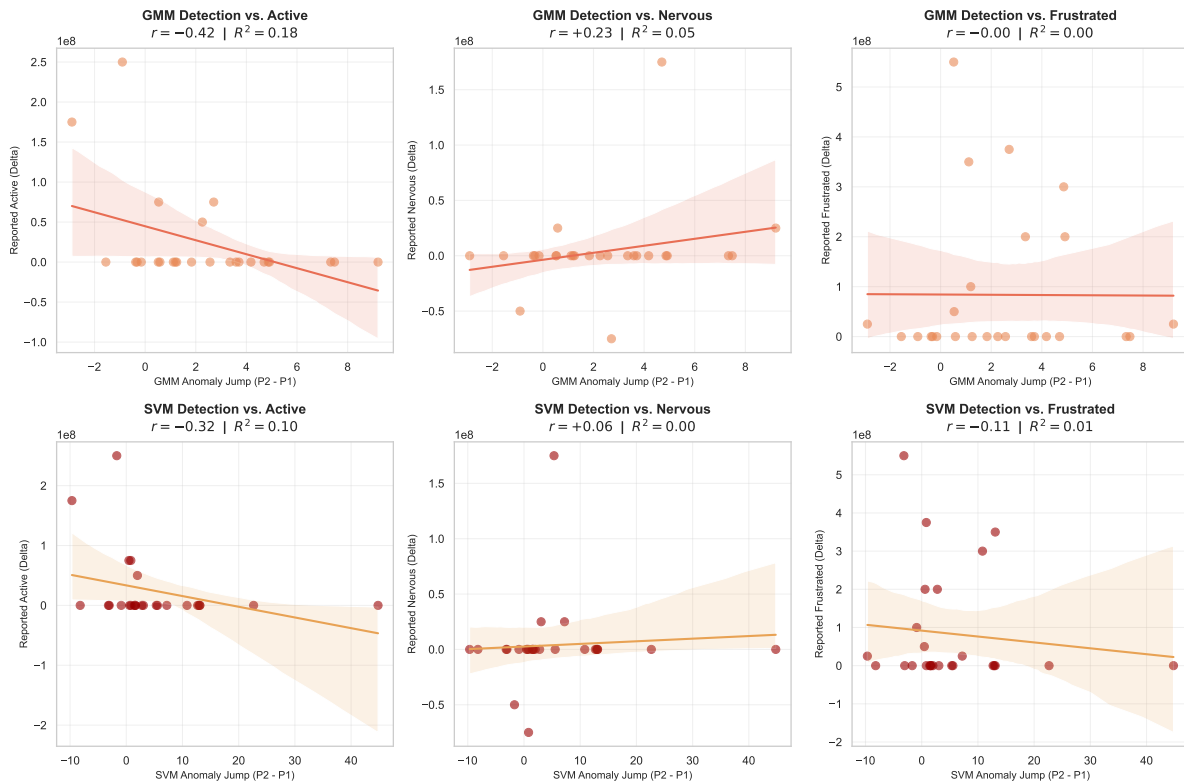


Figure 3: Per-subject correlation between the model's anomaly score jump (P2 - P1) and the change in self-reported emotion (P2 - P1) for three response categories. Top row: GMM. Bottom row: OC-SVM. Each dot is one subject; the shaded band is the 95% confidence interval of the linear fit.

AI-aid use declaration

need to be written

1 Must be in report:

Written Report – Organization & Structure: *Layout is organized, effective, professional, and follows the provided template * Clarity of content (clear research questions and transparent methodology)

– Written Report – Technical Solution: * Design and justification of the ML-pipeline (Data Pre-processing → Representation → Evaluation)

* Robustness of data handling (Clear justification for how missing values, scaling, and specific data types were handled)

* Validity of model validation (Correct methodology for estimating generalization error/EPE, appropriate data splits)

* Critical assessment of data quality, signal strength, and model limitations (e.g., properly diagnosing a 'null result' due to noisy data)

* Originality, rigor, and complexity of method (Note: Rigorous evaluation of a simple method is valued as highly as a complex method)

* Quality of results (How well does the method solve the problem, or how well are the limitations/weak signals explained?)

* The method supports the main points (the aim)

* Reproducibility and code quality (Repository is linked, code is readable and organized)

2 what we need to do

- SVM
 - A (bar) plot from the SVM
 - Some numbers about performace SVM
 - a table of the gammas and mus we use (gridsearch)
- what happens to the missing values
- read through the evaluation pdf so we know if we are lacking
- write about GMM
 - rewrite the introduction to fit the GMM and SVM
 - add small paragraph about PCA in preprocessing
 - add a plot of performance to compare with SVM
- poster is still missing model, results

References

- [1] scikit-learn developers, *Gaussian mixture models — estimation algorithm: Expectation-maximization*, <https://scikit-learn.org/stable/modules/mixture.html#estimation-algorithm-expectation-maximization>, Accessed 2026-05-03.